

CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 01/Unit 01: Introduction

Tauqeer Saleem

Slides based on Dr Faisal Alvi's slides

For any suggested modifications please email: tauqeer.saleem@sse.habib.edu.pk

Introduction



Tauqeer Saleem

توقیر سلیم



Syllabus

1. Lets go to Canvas and take a quick look at the syllabus
 1. Assessments
 2. Grading Scale
 3. Tentative Weekly Schedule
 4. Attendance Policy
 5. Final Exam – TBD
 6. Academic Integrity



Class logistics

1. Please make a nametag and place it in front of your desk
 1. Nothing fancy, you can fold a piece of paper and write your name on it.
 2. I would really appreciate if you could add your pronouns and preferred name.
2. We will be taking short quizzes just for class participation (NOT graded)
3. I expect you to bring your laptops (fully charged) and code along and practice within class.



Unit Outline

1. A Gentle Introduction to Object Oriented Programming
2. C++ and hello world!
3. GNU Tool chain
4. Variables and Data Types
5. Input and Output
6. Loops
7. Functions
8. Summary

A Gentle Introduction to Object Oriented Programming

What is OOP? How is it different from what you have been
doing so far?

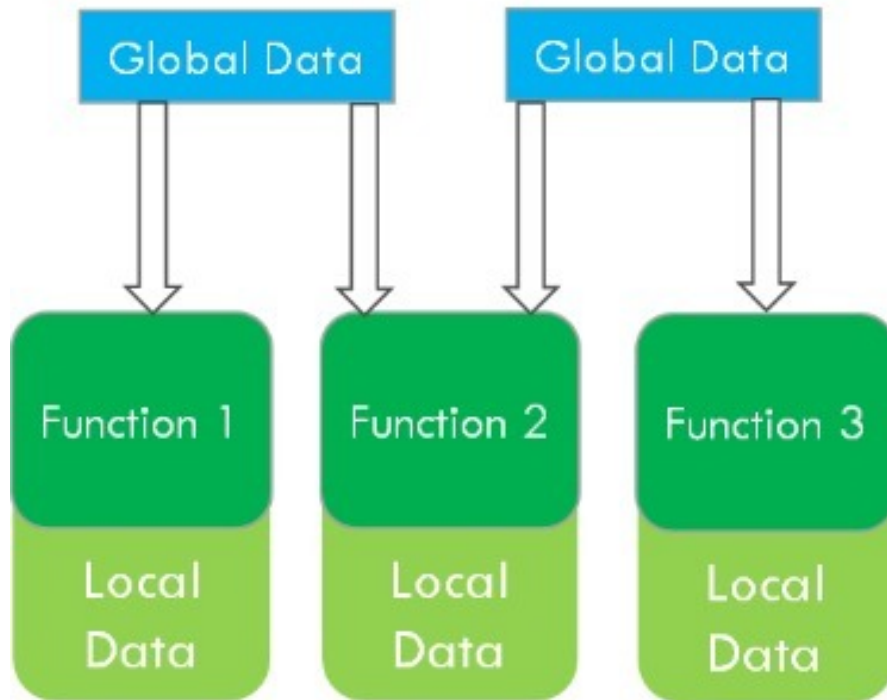


What is Object Oriented Programming?

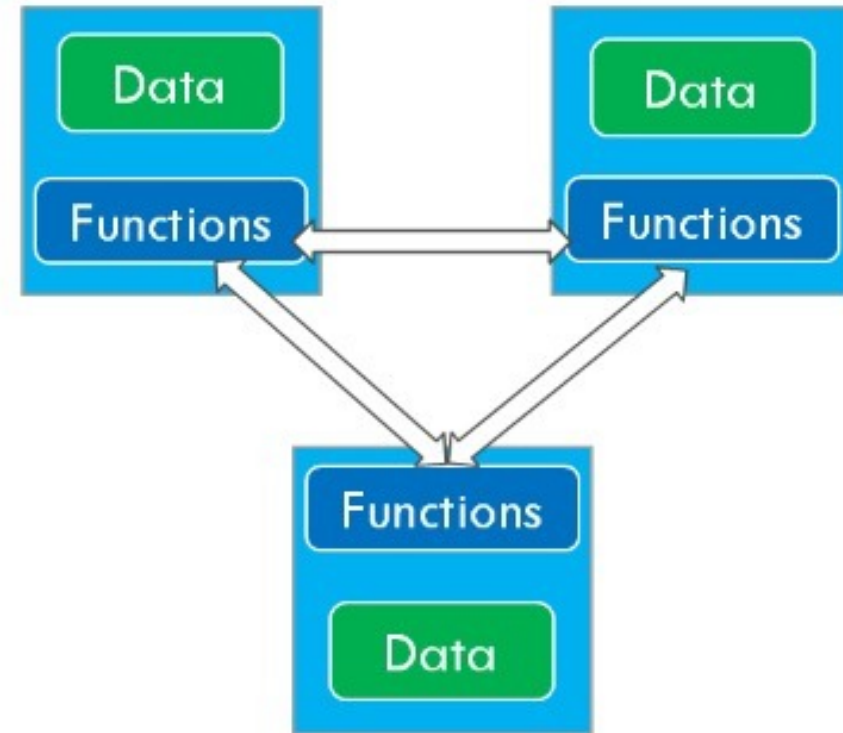
- Object-oriented programming (OOP) is a programming paradigm based on the concept of objects, which can contain **data** and **code**:
 - **data** in the form of fields (often known as *attributes* or *properties*), and
 - **code** in the form of procedures or functions (often known as *methods*).
- In OOP, computer programs are designed by creating objects that interact with one another.

Procedural VS Object Oriented Programming

Procedural Oriented Programming



Object Oriented Programming



First steps create the path for great strides

We will first learn basic C++ and then come back again to OOP concepts and applications in C++

C++

What is C++? Why are we learning it?

C++

C++ is a high-level, general-purpose programming language created by Danish computer scientist Bjarne Stroustrup.



1. 

2.  java

3.

4.

5.

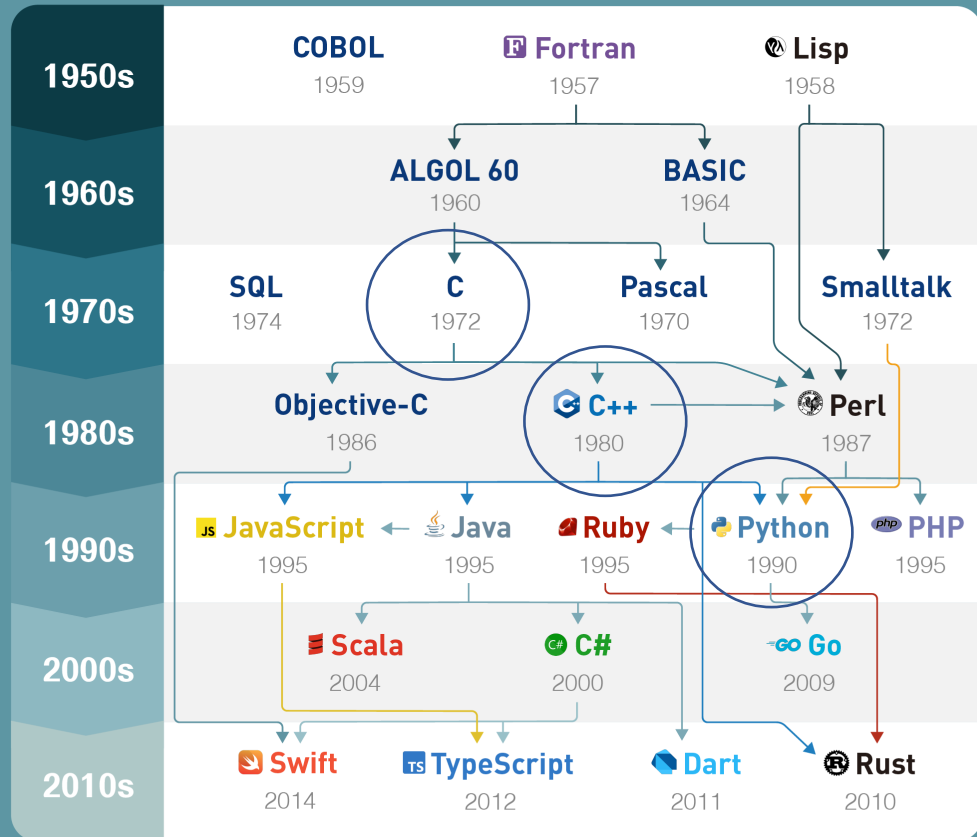




C++



Timeline of Programming Languages



Features not used in
C++

C

C+

+

C++

Several standards; for this course, we use C++ 17.

C++ standards

Year	ISO/IEC Standard	Informal name
1998	14882:1998 ^[34]	C++98
2003	14882:2003 ^[35]	C++03
2011	14882:2011 ^[36]	C++11, C++0x
2014	14882:2014 ^[37]	C++14, C++1y
2017	14882:2017 ^[38]	C++17, C++1z
2020	14882:2020 ^[17]	C++20, C++2a
TBA	14882:2024	C++23, C++2b
TBA		C++26, C++2c

Writing our first program!

Hello world!

Hello World – The anatomy of C++ code

```
1  #include <iostream>
```

Preprocessor Directive; Include files

```
2
```

```
3  using namespace std;
```

Using namespace

```
4
```

```
5  int main(void) {
```

Main function (observe the braces)

```
6      cout << "Hello World" << endl;
```

Cout (stands for console-output) with insertion operator

```
7      return 0;
```

<< Return value (final output of the function main)

```
8  }
```




Preprocessor Directives

- `#include <iostream>` and other statements not seen in the figure before (`#define`, `#undef`, `#if`, `#elif`, `#else`, `#endif`, `#ifdef`)
- `#include`: Instructs the preprocessor to replace “`#include <XYZ>`” by the content of the file “XYZ”
 - In this case it's a standard C++ code, known as header `iostream`, that allows to perform standard input and output operations.

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main(void) {
6      cout << "Hello World" << endl;
7      return 0;
8  }
```



Preprocessor Directives

- You might encounter something like `#define VALUE 42`
- This basically replaces `VALUE` with 42 everywhere in the your code
- Remember this is just text replacement, so its not type safe and cause weird bugs

```
#define VALUE 42
int main() {
    return VALUE;
}
```



Preprocessor Directives - Quiz

```
#define PI 3.141

int main(){
    int r = 1;
    double area_circle = PI * r * r;
    return 0;
}
```

Can you tell what will be the final expression for area_circle after the preprocessor replaces PI?



Preprocessor Directives - Quiz

```
#define square(x) x*x

int main(){
    int length = 1;
    double area_square = square(length + 1);
    return 0;
}
```

Can you tell what will be the final expression for area_square after the preprocessor replaces PI?



"Using" Directive

- C++ program can be divided into different namespaces. A namespace defines the scope of certain names; outside of the namespace they're unknown.
- The directive "using namespace std;"
 - says that all the program statements that follow are within the std namespace.
- Various program components such as cout are declared within this namespace.

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main(void) {
6      cout << "Hello World" << endl;
7      return 0;
8  }
```



int main()

- Multiple variations
- Subtle differences
- But lets focus on what it is before going into these differences

```
#include <iostream>
using namespace std;

int main(int argc, char* argv[]){
    cout << "Hello World!" << endl;
    return 0;
}
```

```
#include <iostream>
using namespace std;

int main(void){
    cout << "Hello World!" << endl;
    return 0;
}
```

```
#include <iostream>
using namespace std;

int main(){
    cout << "Hello World!" << endl;
    return 0;
}
```



int main()

int main()

- **int** indicates the return type of the function.
- The function named main is a special function in all C++ programs; it is the function called when the program is run.
- The execution of all C++ programs begins with the main function, regardless of where the function is actually located within the code.

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main(void) {
6      cout << "Hello World" << endl;
7      return 0;
8  }
```



int main(void)

```
#include <iostream>
using namespace std;

int main(void){
    cout << "Hello World!" << endl;
    return 0;
}
```

void just indicates that this special function (main) does not take
any inputs



int main(int argc, char* argv[])

```
#include <iostream>
using namespace std;

int main(int argc, char* argv){
    cout << "Hello World!" << endl;
    return 0;
}
```

int argc, char* argv[] are inputs provided by the terminal (not needed in every program)



int main()

```
#include <iostream>
using namespace std;

int main(){
    cout << "Hello World!" << endl;
    return 0;
}
```

In fact, this is what you will use 99.9% of the times in this course int main(), so don't worry about the other variants

The C++ compiler : g++

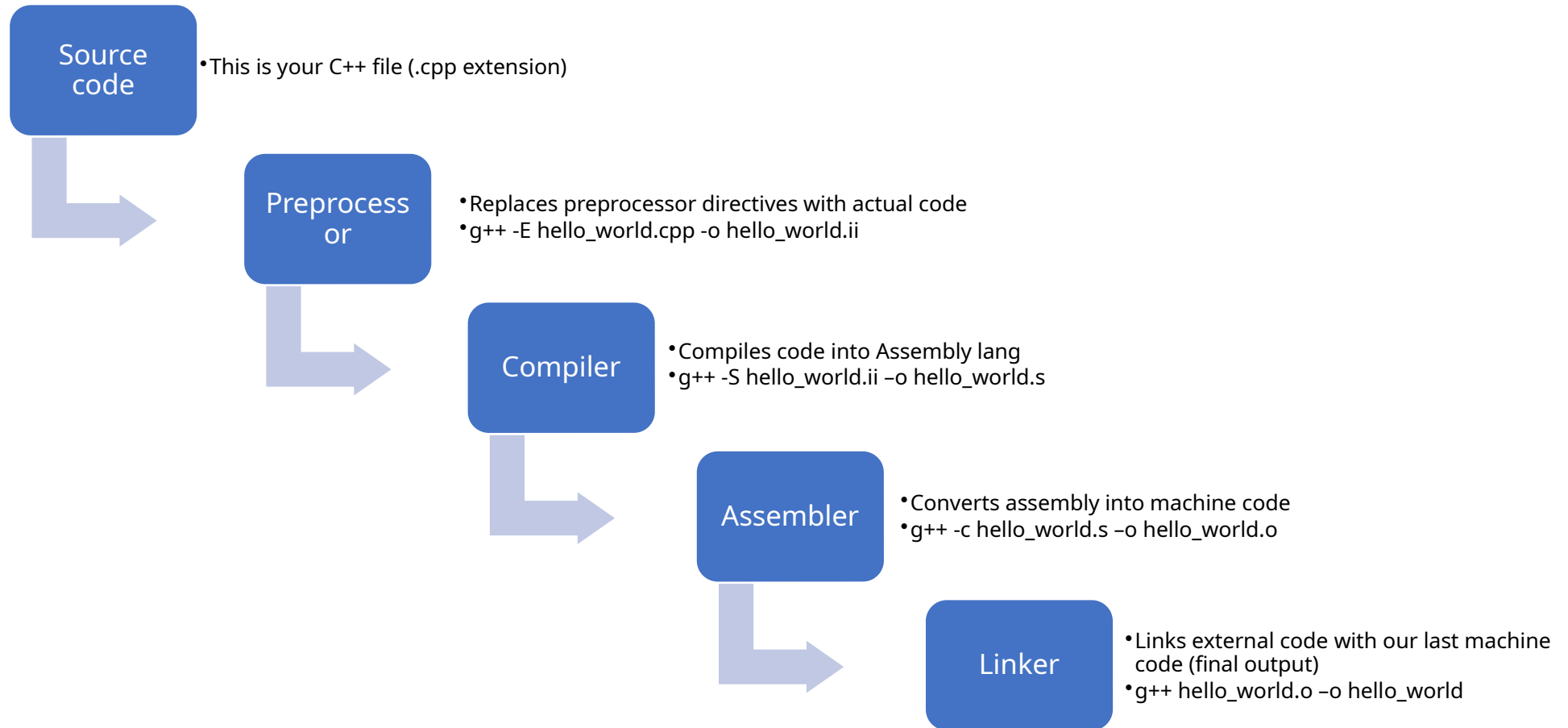
Preprocessor.....Compiler...Assembler...Linker
GNU tool chain



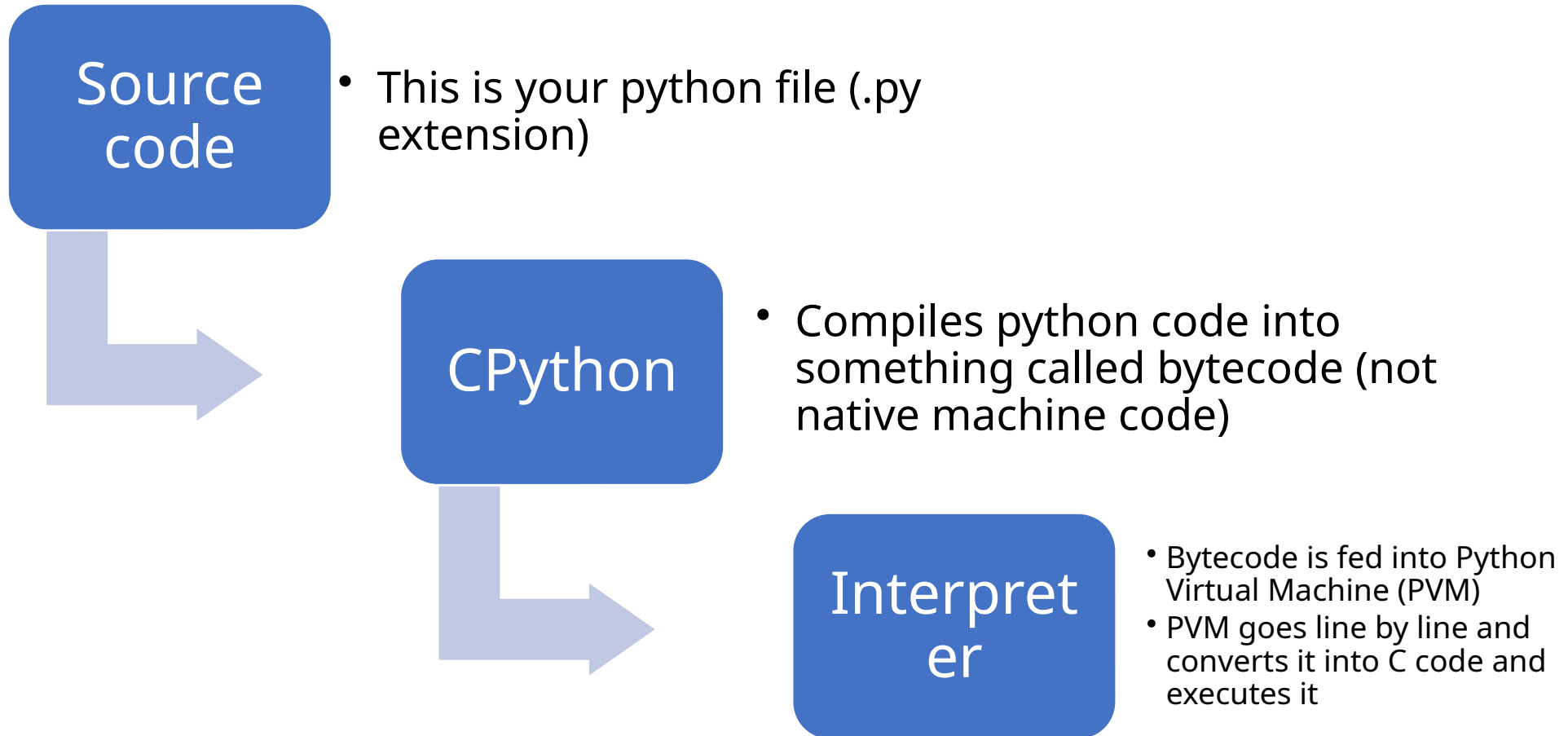
A short story about GNU

- GNU stands for “GNU Not Unix”... did you get the joke?
- It's a project started in 1983 by Richard Stallman set out to build a unix-like operating system which is opensource and free.
- Since then it has incorporated a lot of other software.
- One of them is g++, short for GNU C++.
- Its not just a single tool, it is a connection – a chain – of multiple tools that connect to each other
- So it is also called GNU tool chain

The GNU tool chain



How is this different from Python?





int main() - Quiz

```
#define VALUE 42
#define MESSAGE "Hello!"
int main() {
    return VALUE;
}
```

- Lets code along the code shown above
- Then go to terminal and run “g++ -E -P hello_world.cpp -o hello_world.ii
- Then go to terminal and run “g++ -S hello_world.cpp -o hello_world.s”

Data types

And why do we need to specify them in C++? And why not in python?



How C++ stores values?

- A **variable** is a memory address where data can be **stored** and **changed**.
- Declaring a variable means specifying both its **name** and its **data type**.
- A declaration tells the compiler to **allocate enough memory** to hold a value of this data type, and to **associate the identifier** with this location.
- Different variables will have different memory size, so to correctly use this memory address, we need to tell C++ .



Primitive Data Types in C++

- **Integral Types**

- represent whole numbers and their negatives
- declared as int, short, or long

- **Character Types**

- represent single characters
- declared as char
- Stored by ASCII values



Primitive Data Types in C++

- **Boolean Type**
 - declared as bool
 - has only 2 values true/false
 - will not print out directly
- **Floating Types**
 - represent real numbers with a decimal point
 - declared as float, or double
 - Scientific notation where e (or E) stand for "times 10 to the " (.55-e6)



Primitive Data Types in C++

Type	Numeric range	Digits of precision	Typical size
bool	false (0) or true (1)	N/A	1 byte
char	-128 to 127 (signed char) 0 to 255 (unsigned char)	Exact integer	1 byte
short	-32,768 to 32,767	Exact integer	2 bytes
int	-2,147,483,648 to 2,147,483,647 Different based on OS: Linux: -2,147,483,648 to 2,147,483,647 Windows: 9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	Exact integer	4 bytes 4 bytes (Windows) 8 bytes (Linux)
long	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	Exact integer	8 bytes
float	approx $\pm 3.4 \times 10^{38}$	6 - 7 decimal digits	4 bytes
double	apporx $\pm 1.7 \times 10^{308}$	15 - 16 decimal digits	8 bytes



How C++ stores values?

- Lets focus on the white board now!



Type Conversions

- Handled automatically

```
int count = 7;  
float avgWeight = 155.5F;  
double totalWeight = count * avgWeight;
```

- When two operands of different types are encountered in the same expression, the lower-type variable is converted to the type of the higher-type variable.
- Order of types from highest to lowest:
long double > double > float > long > int > short > char



Type Conversions (Casting)

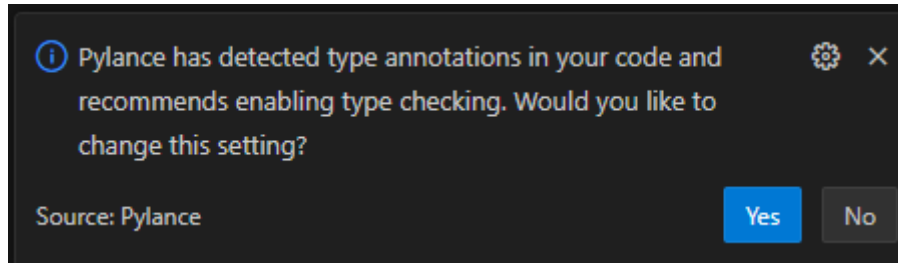
- Casting - forcing conversion by putting (type) in front of variable or expression. Used to insure that result is of desired type.
- Example: If you want to divide two integers and get a real result you must cast one to double so that a real divide occurs and store the result in a double.
- `int x = 5, y = 2; double z;    z = static_cast <double> (x)/y;    //`
2.5
- `int x = 5, y = 2; double z;    z = (double) x/y;    //`
2.5
- `int x = 5, y = 2; double z;    z = static_cast <double> (x/y);    //`
2.0
- `static_cast<int> (z)` - will truncate z
- `static_cast <int> (x + 0.5)` - will round positive x {use () to cast complete expression}



Type Conversions (Casting) - Quiz

- Would `static_cast<int> (x + 0.99999999999)` work as a rounding UP method?

Did you know

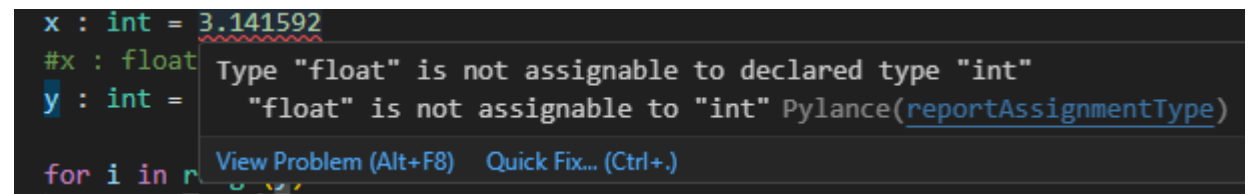


Python > Analysis: **Type Checking Mode**
Type checking modes Basic, Standard, and Strict :

Feature	Basic	Standard	Strict
Variable type mismatches	✓	✓	✓
Function return type checks	✓	✓	✓
Type narrowing enforcement		✓	✓
Checking of <code>Any</code> type		✓	✓
Private/protected access checks		✓	✓
Enforces stricter generics usage		✓	✓
Reports missing type annotations			✓
Disallows <code>Any</code> type usage			✓
Requires strict type compatibility			✓
Enforces complete type coverage			✓

For more details, check the [Pylance documentation](#).

- In python, advance programmers used “type annotations” to specify data types
- So we have something similar in python too!



Input & Output

What is “cout”? Why a line in c++ is so ugly?



Input/Output

- The identifier `cout` (pronounced “C out”) is actually an object. It is predefined in C++ to correspond to the standard output stream.
- A stream is an abstraction that refers to a flow of data.
- The standard output stream normally flows to the screen display—although it can be redirected to other output devices.
- The operator `<<` is called the **insertion** or **put to** operator. It directs the contents of the variable on its right to the object on its left.
- Example: Refer to the “HelloWorld” Program.



Input/Output

- The keyword `cin` (pronounced “C in”) is an object, predefined in C++ to correspond to the standard input stream.
- This stream represents data coming from the keyboard (unless it has been redirected).
- The `>>` is the **extraction** or **get from** operator.
- It takes the value from the stream object on its left and places it in the variable on its right.



Input/Output

- Example usage

```
int main(){  
    int x;  
    int y;  
    cin >> x;  
    cin >> y;  
    cout << x + y << endl;  
}
```

Arithmetic Operator

What is “cout”? Why a line in c++ is so ugly?



Arithmetic Operators

- Operators: +, -, * /
- For floating numbers, the result is same as Math operations.
- Note on integer division: the result is an integer. $7/2$ is 3.
- % (remainder or modulo) is the special operator just for integer. It yields an integer as the result. $7\%2$ is 1.
- Both / and % can only be used for positive integers.
- Precedence rule is similar to Math.



Relational Operators

- Expression that yields bool result
- Include:
- 6 Relational Operators

< <= > >= == !=

- 3 Logical Operators

! && ||



Relational Operators - Quiz

- `Int x; cin >> x;`
- `Int y; cin >> y;`
- `int z = 10*static_cast<int>(!(x>=y)) + 5*static_cast<int>(!(x<=y))`
- `Cout << z << endl;`

- `int z = 10*(x<y) + 5*(x>y)`
- Will this work or give an error because x, y are bools

Control flow statements

Loops and conditionals

Decisions

- if-else statements
- Similar to Python with some syntactic differences:

```
# Compare and find the maximum value
if a >= b and a >= c:
    max_value = a
elif b >= a and b >= c:
    max_value = b
else:
    max_value = c
```

```
// Compare and find the maximum value
if (a >= b && a >= c) {
    max = a;
} else if (b >= a && b >= c) {
    max = b;
} else {
    max = c;
}
```

Loops

- while, do-while and for-loops
- Similar to Python with some syntactic differences (e.g. for-loop)

```
for i in range(1, 11):  
    print(i, end=" ")
```

```
// Loop from 1 to 10  
for (int i = 1; i <= 10; ++i) {  
    std::cout << i << " ";  
}
```

- Question to Explore: What is ++i? Is it different to i++?



Control flow statements - Quiz

- What is the output?

```
for(int i = 0; i < 5; i++){  
    for(int j = 0; j < i; j++) cout << "*";  
    cout << endl;  
}
```



Control flow statements - Quiz

- What is the output?

```
for(int i = 0; i < 5; i++){  
    for(int j = i; j > 0; j--) cout << "*";  
    cout << endl;  
}
```

Control flow statements - Quiz

- What is the output?

```
for(int i = 0; i < 5; i++){  
    for(int j = i; j > 0; j--) cout << "*";  
    cout << endl;  
}  
  
for(int i = 0; i < 5; i++){  
    for(int j = 5 - i; j > 0; j--) cout << "*";  
    cout << endl;  
}
```



Quiz

```
#include <iostream>

int main(int argc, char* argv[]){
    for(int i = 0; i < argc; i++){           // consider this a loop over the list argv
        std::cout << argv[i] << std::endl; // this is just print(x)
    }
}
```


Functions

Arguments vs parameters, return type, functions in python vs C++



Functions

- Functions in C are similar to those in Python with one major difference.
- Every function must have a return type, and each parameter (if present) must have a data type.
- For example in Python we can define a function increment which increments the variable

```
def increment(x):  
    return x + 1
```

- In C++, this will be: `int increment(int x)`

```
int increment(int x) {  
    return x + 1;  
}
```



Summary

- In this session we covered the basics of C++ Programming.
- Specifically we covered data types, type conversions, Arithmetic and Relational Expressions, conditional operators and loops.
- In the subsequent sessions, we will use these to build programs.

Class Exercise: Write a program in C++ that reads in the coefficients a , b and c of a quadratic equation $ax^2 + bx + c = 0$ and finds and prints the roots of the equation.

Bonus: Write the roots of equations that are complex in the proper form.